

Module 10 — Reflection Document

IS 601 — Python for Web API | NJIT | April 2026
Secure User Model, CI/CD Pipeline, and Docker Hub Deployment

GitHub Repository

GitHub Repository Link:

```
https://github.com/YOUR_USERNAME/module10-secure-user-api
```

Docker Hub Repository Link:

```
https://hub.docker.com/r/niharika2701/module10-secure-user-api
```

Reflection

This module introduced me to building a secure user model in FastAPI using SQLAlchemy and Pydantic, and then automating the full test and deployment cycle with GitHub Actions and Docker Hub.

SQLAlchemy and Pydantic

Setting up the SQLAlchemy User model felt straightforward once I understood how DeclarativeBase works. Defining columns for username, email, and password_hash with unique constraints made sense from a data integrity standpoint. The Pydantic schemas (UserCreate and UserRead) were equally important. Having separate schemas for input and output meant that password_hash never gets returned to the client, which is a basic but critical security principle. I had not thought about that separation before.

Password Hashing with passlib and bcrypt

Using passlib with the bcrypt scheme made hashing simple. One thing that surprised me was that hashing the same password twice produces different results because of the random salt. This is intentional and makes the system more secure against attacks like rainbow tables. Writing unit tests specifically for the hash and verify functions helped me understand this behavior clearly.

Testing with pytest and SQLite

Running tests locally against SQLite instead of PostgreSQL was a practical decision. psycopg2-binary does not support Python 3.13 yet, so using SQLite for local tests and PostgreSQL only in GitHub Actions was a good workaround. The conftest.py fixture pattern with dependency injection (overriding get_db) was new to me and I found it very clean. Each test function gets a fresh database, which prevents tests from affecting each other.

CI/CD with GitHub Actions and Docker Hub

The GitHub Actions workflow was the most interesting part. The pipeline has two jobs: test and deploy. The deploy job only runs if the test job passes, and only on pushes to main. This is a real-world pattern that prevents broken code from being deployed. Using a Docker Hub access token instead of a password for authentication was also a good security practice. Storing credentials as GitHub Secrets kept them out of the codebase entirely.

Challenges

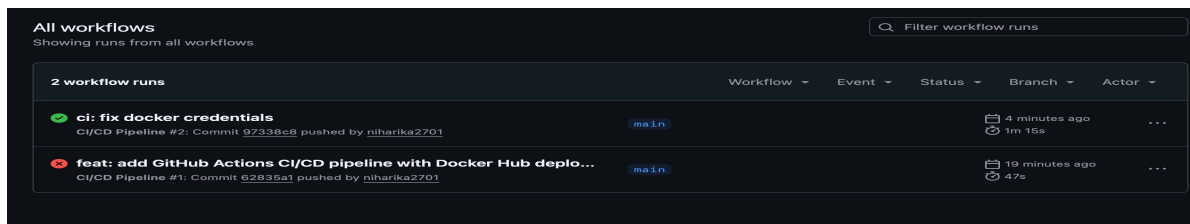
The biggest challenge was package compatibility. SQLAlchemy 2.0.23 and psycpg2-binary 2.9.9 both had issues with Python 3.13 on my Mac M4. Splitting requirements into requirements.txt for Docker and requirements-dev.txt for local testing solved this. The Docker Hub login also failed initially because GitHub Secrets had the wrong credentials. Switching to an access token fixed it immediately.

Looking Ahead

This module laid the foundation for the final project. Understanding how the User model, schemas, hashing, tests, and CI/CD pipeline connect together gives me a clearer picture of what a production-ready API looks like. Future modules covering database migrations with Alembic and JWT authentication will build directly on this setup.

Screenshots

Screenshot 1 — GitHub Actions Workflow



Caption: GitHub Actions CI/CD Pipeline showing 2 workflow runs. The most recent run "ci: fix docker credentials" completed successfully in 1 minute 15 seconds, confirming all tests passed and the Docker image was pushed to Docker Hub.

Screenshot 2 — Docker Hub Repository

niharika2701/module10-secure-user-api

Last pushed 3 minutes ago · 0 · 0

Add a description

Add a category

General

Tags

Image Management

Collaborators

Webhooks

Settings

Tags

DOCKER SCOUT INACTIVE

Activate

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	3 minutes

See all

Caption: Docker Hub repository niharika2701/module10-secure-user-api showing the latest image tag pushed 3 minutes ago by the GitHub Actions deploy job. The image is publicly accessible and ready to pull.